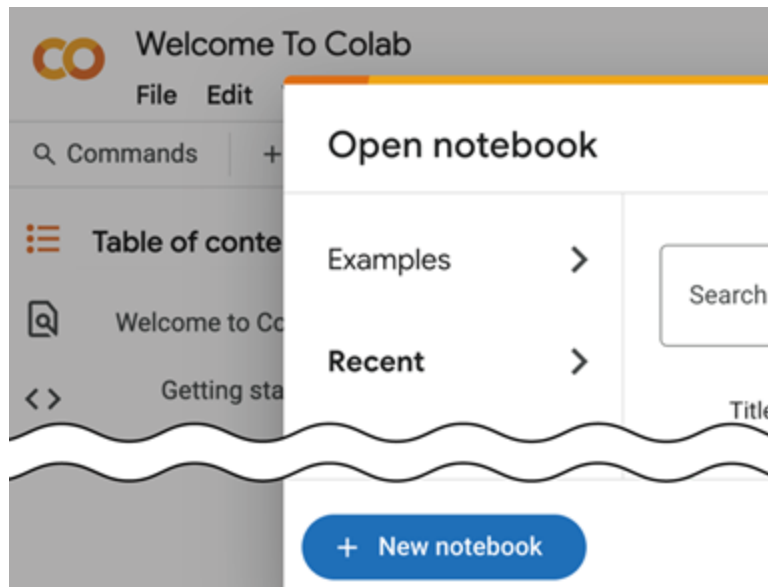


2. Basics of Python

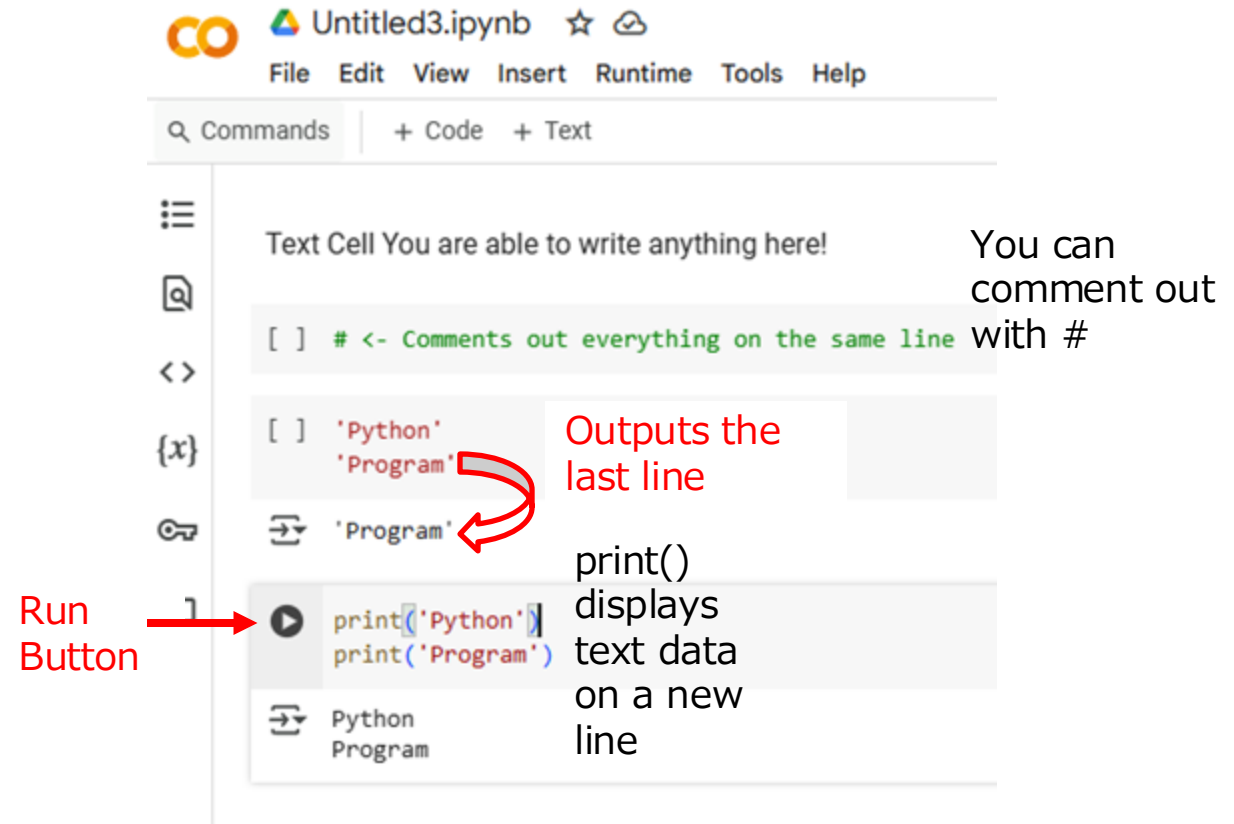
Python is a programming language with various applications, including data science.

Data Science × Python

- Basic data science implementation
(e.g. Model Training, Model Evaluation)
- Web Scrapping
- Available on Google Colaboratory
(<https://colab.research.google.com/>)



Code on Google Colab



Python programs are mainly composed of **words, symbols, and numbers**.

Example Python Code

*Do not worry about the meaning of code

Code input

In [1]: 'Python Program'

Out [1]: 'Python Program'

Output

In [2]:
score = [1, 2, 3]
average = sum(score) / len(score)
average > 1

Out [2]: True

Read Them Out Like This!

Words: **print**, **sum**, **len**

Symbols: **()** "Parenthesis", **[]** "square brackets", **,** "comma", **/** "forward slash", **'** "single quotation mark"

Score equals open square bracket 1 comma 2 comma 3 close square bracket

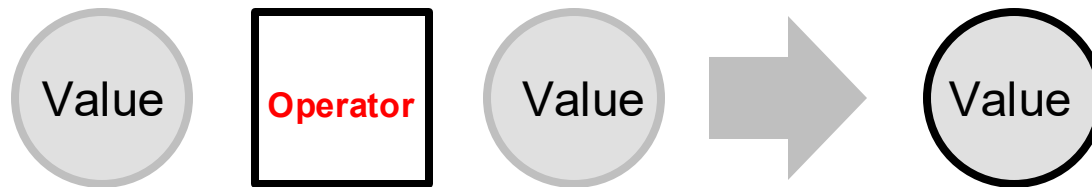
Average equals sum open parenthesis score close parenthesis divided by len open parenthesis score close parenthesis

Average greater than 1

Parentheses and brackets are important symbols as they can not be changed or omitted.
Names like score and average are strings and can be freely changed.

An **operator** performs a calculation on values and returns the result.

Conceptual Diagram



e.g. Arithmetic Operators: Addition



Example Code

```
In [1]: 1 + 1
Out[1]: 2
In [2]: 1 - 1
Out[2]: 0
```

Input to a code cell

Displayed output

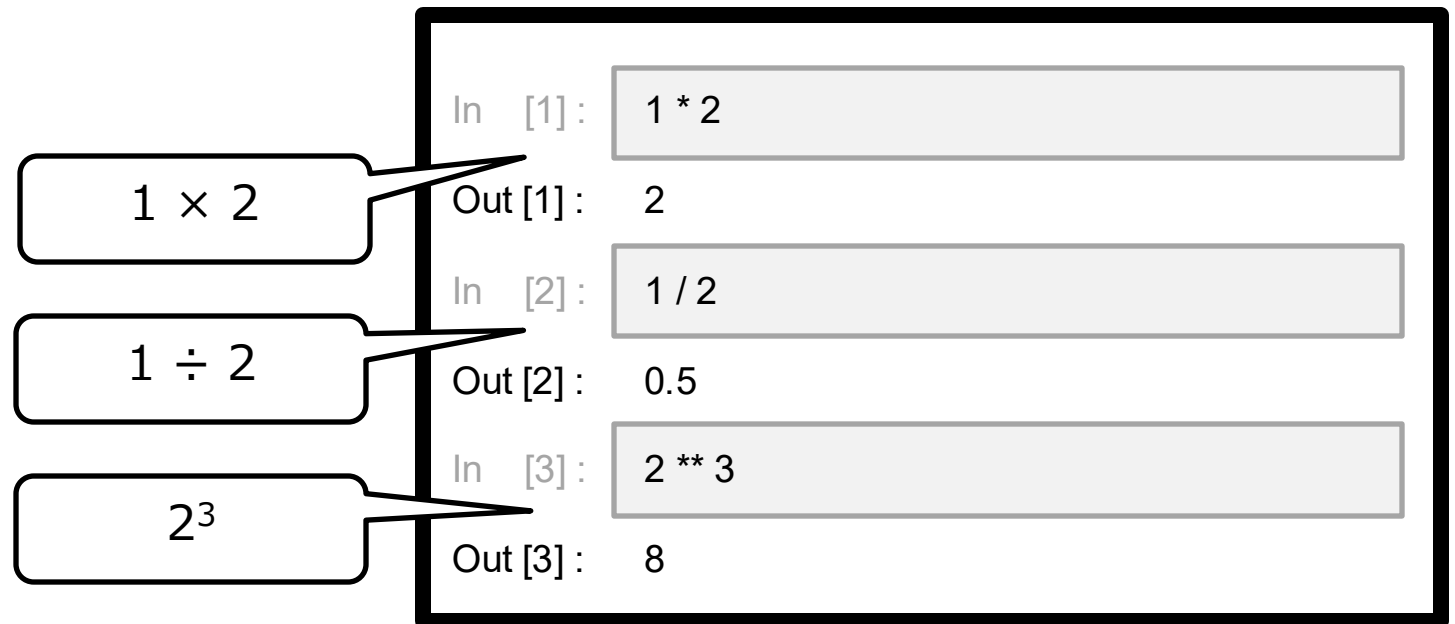
Addition, subtraction: same symbols as in math

Multiplication, division, and exponentiation: unique symbols

Example Arithmetic Operators

Arithmetic Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponentiation

Example Code



Introduction to Data Science

Comparison Operator



An **comparison operator** is a symbol used to compare the values on the left and right. The result is evaluated as either True or False.

Example Comparison Operators

Comparison Operator	Operation
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to each other
!=	Not equal to

Example Code

```
In [1]: 1 > 0
Out[1]: True
In [2]: 1 == 0
Out[2]: False
```

"=" signs × 2

e.g. Comparison Operator:

> Greater Than



Quiz - Check Your Understanding



Fill in the blanks.

Arithmetic Operators

In [1]: $3 \square 2$

Out [1]: 5

In [2]: $3 \square 2$

Out [2]: 6

In [3]: $3 \square 2$

Out [3]: 1.5

Comparison Operators

In [1]: $1 < 1$

Out [1]:

In [2]: $1 \geq 1$

Out [2]:

In [3]: $1 \neq 0$

Out [3]:

Quiz - Check Your Understanding



Answers

If you made a mistake, review the table of arithmetic and comparison operators.

Arithmetic Operators

In [1]: $3 + 2$

Out [1]: 5

In [2]: $3 * 2$

3×2

Out [2]: 6

In [3]: $3 / 2$

$3 \div 2$

Out [3]: 1.5

Comparison Operators

In [1]: $1 < 1$

Out [1]: False

In [2]: $1 \geq 1$

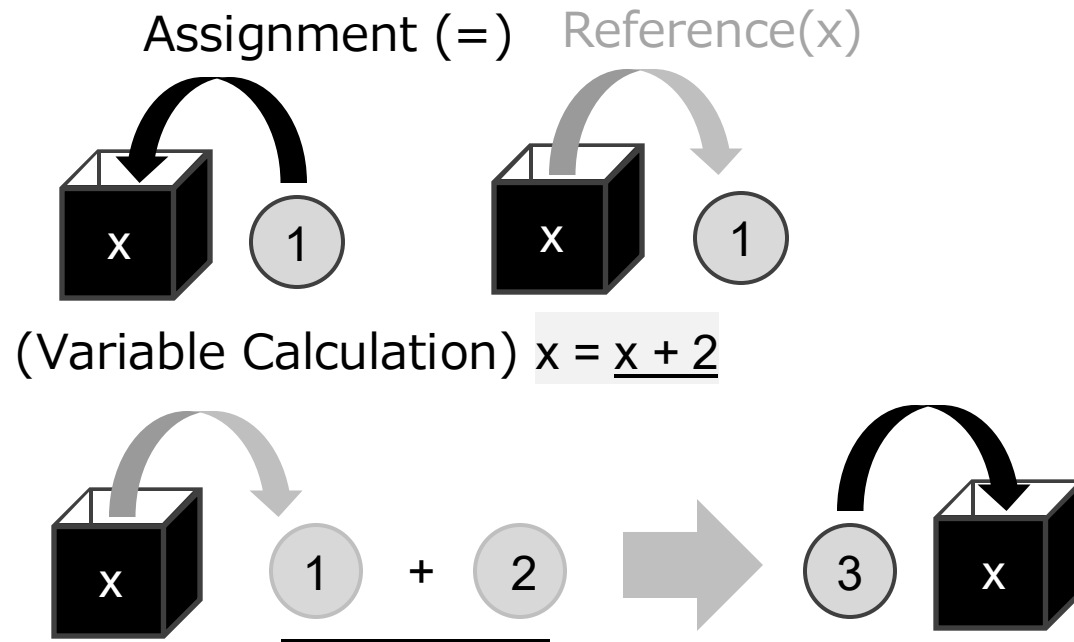
Out [2]: True

In [3]: $1 \neq 0$

Out [3]: True

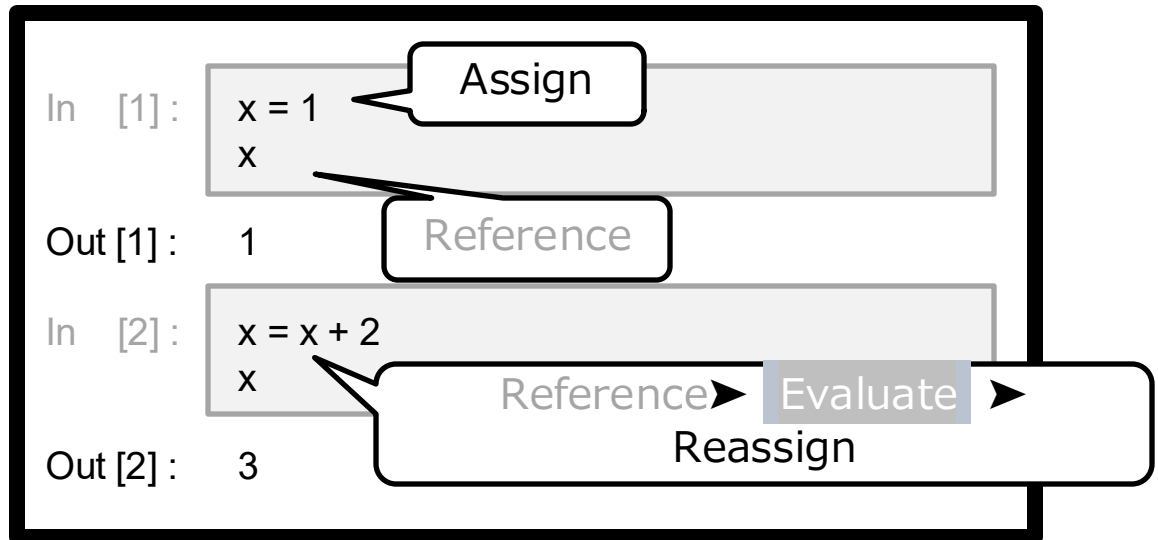
A **variable** is a box where you temporarily store a value with a name.
We **assign** a name like this: **variable_name = value**

Conceptual Diagram



In programming, the = symbol represents assignment, which is different from the equals sign used in mathematics

Example Code



*In assignment statements, the calculation on the right-hand side is performed before the value is assigned to the variable on the left-hand side

You can check a variable's **data type** using the type function.

Common Data Types

Name	Description	e. g.
int	Integer	1
float	Decimal	3.14
str	Text data	'Python'
bool	Boolean	True, False
list	List	[1, 2, 3]

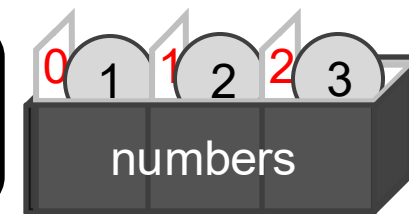
Stores
values in
order

Example Code

```
In [1]: one = '1'
        type(one)

Out[1]: str
```

Input: variable
Output: data type



Quiz - Check Your Understanding



Fill in the blanks.

Example Code (Variables)

```
In [1]: x = 2  
x
```

Out [1]:

```
In [2]: x = x + 1  
x
```

Out [2]:

```
In [3]: type(x)
```

Out [3]:

```
In [4]: y = x / 2  
type(y)
```

Out [4]:

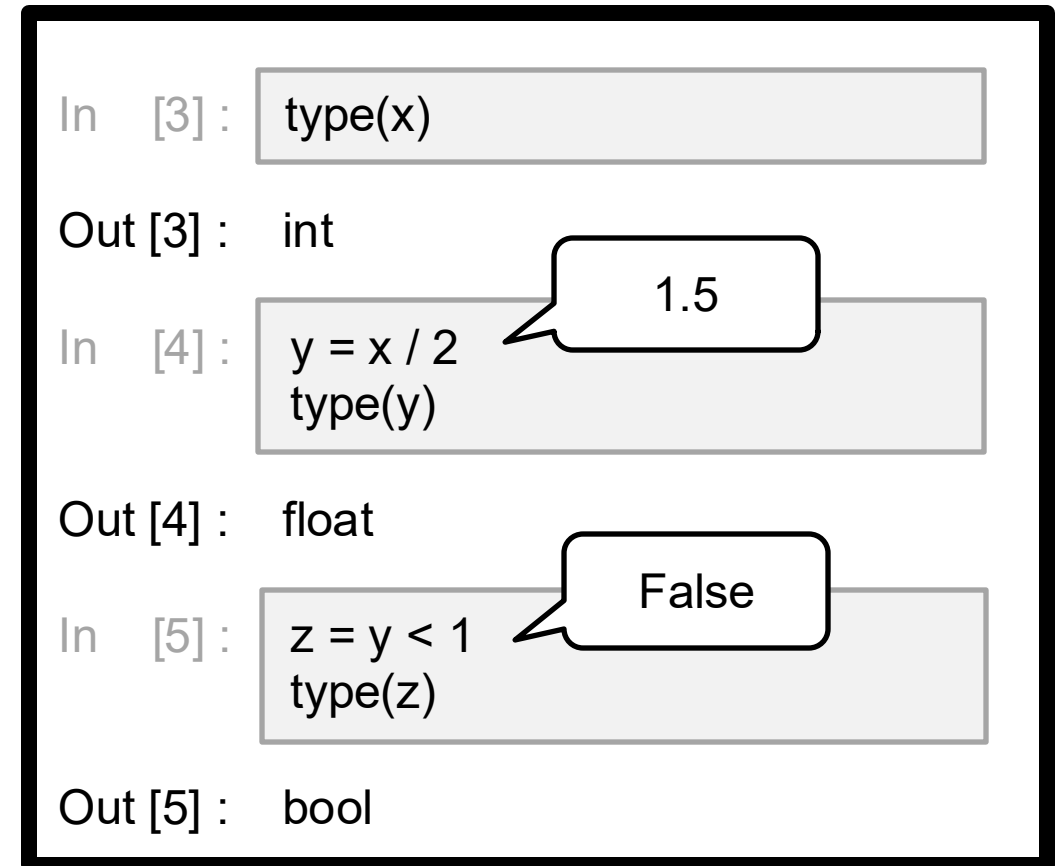
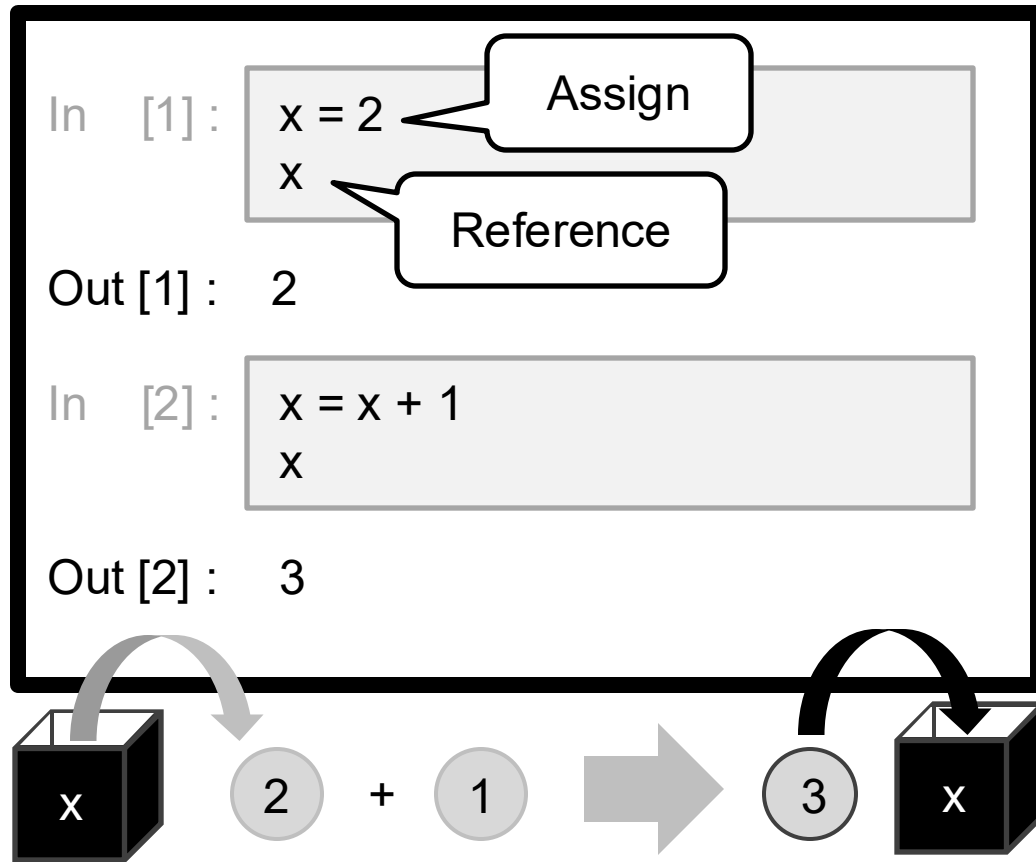
```
In [5]: z = y < 1  
type(z)
```

Out [5]:

Quiz - Check Your Understanding

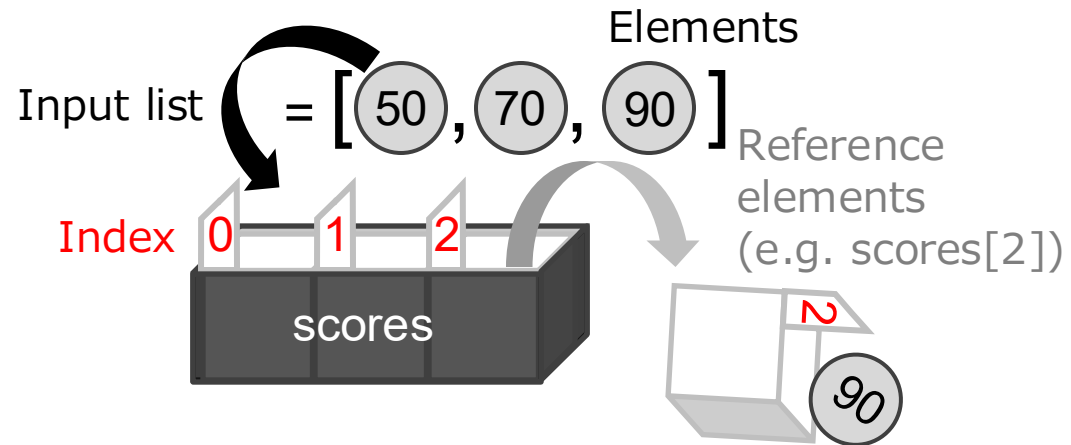
Answers

Example Code (Variables)

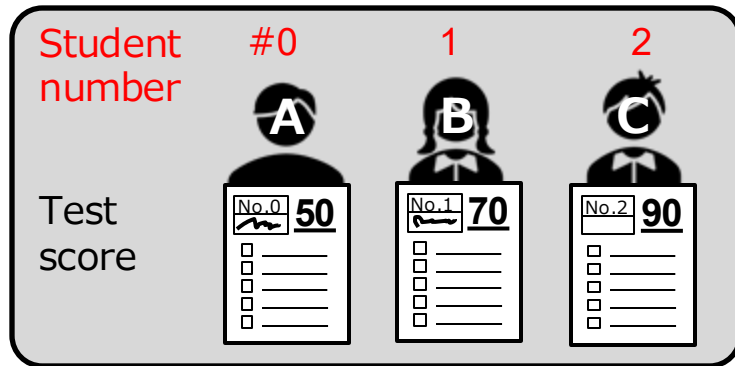


A **list** is a data type that manages multiple values (**elements**) in sequence using **indexes**.

Conceptual Diagram



Example list



Example Code

```
In [1]: scores = [50, 70, 90]
        scores[2]
```

```
Out [1]: 90
```

Assigns score "0" to Index 2

```
In [2]: scores[2] = 0
        scores
```

```
Out [2]: [50, 70, 0]
```

Calculates the sum

```
In [3]: sum(scores)
```

```
Out [3]: 120
```

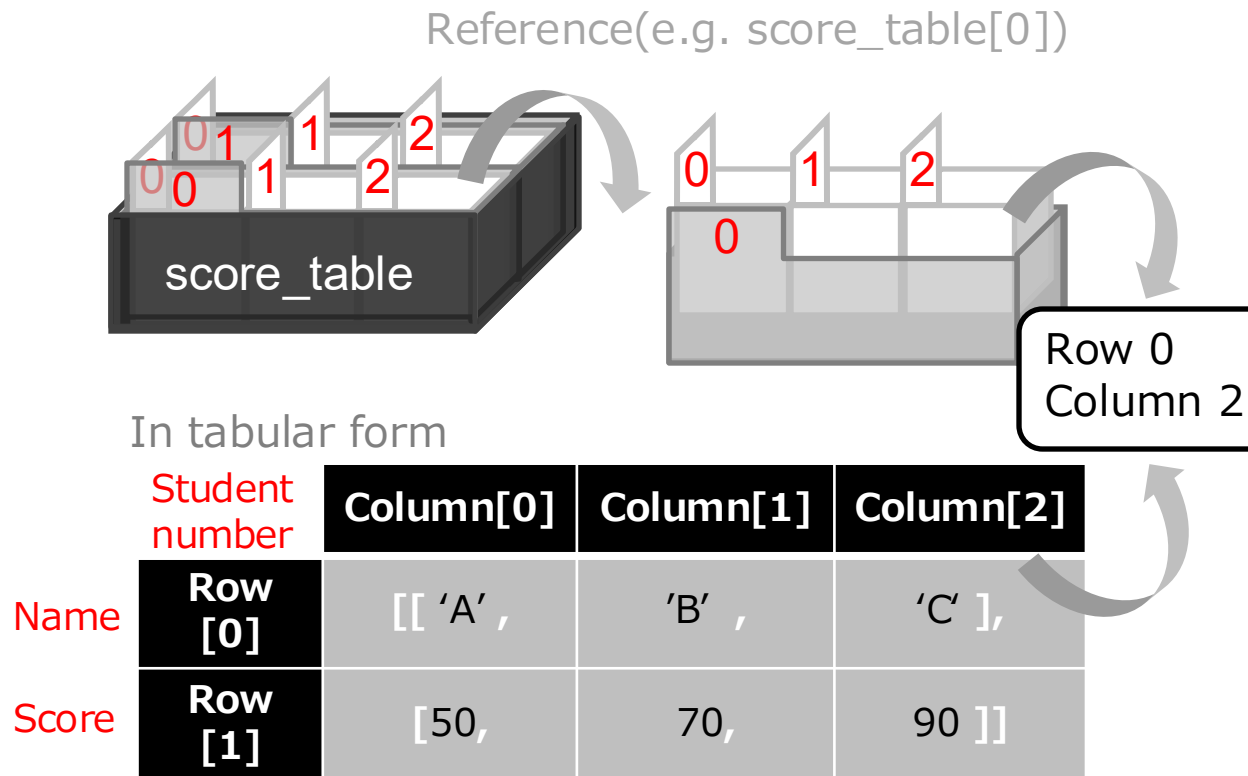
Introduction to Data Science

Two Dimensional List



A list of lists. Manages values in a tabular form. Reference or modify elements by specifying the row and column indices.

Conceptual Diagram



Example Code

```
In [1]: score_table = [ ['A', 'B', 'C'], [50, 70, 90] ]
        score_table[0]

Out [1]: ['A', 'B', 'C']

In [2]: score_table[0][2]

Out [2]: 'C'

In [3]: score_table[1][2]

Out [3]: 90
```

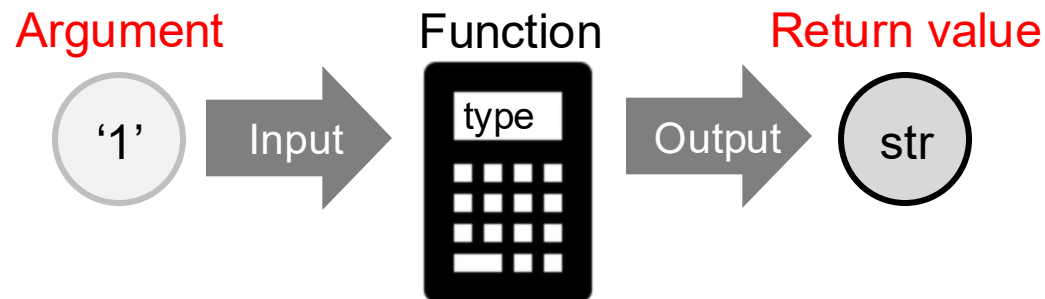
names = ['A', 'B', 'C']
scores = [50, 70, 90]
score_table = [names, scores]

A **function** performs a predefined operation.
Input: "**argument**". Output: "**return value**"

Built-in Functions in Python

Functions	Description
print()	Displays input as text data
type()	Outputs data type
sum()	Outputs the sum

Conceptual Diagram



Example Code

```
In [1]: input_value = '1'
        output_value = type(input_value)
        output_value

Out [1]: str
```

Some functions do not require arguments or return values. Even with no argument, parentheses must be included. Assigning the return value to a variable is not required.

Introduction to Data Science

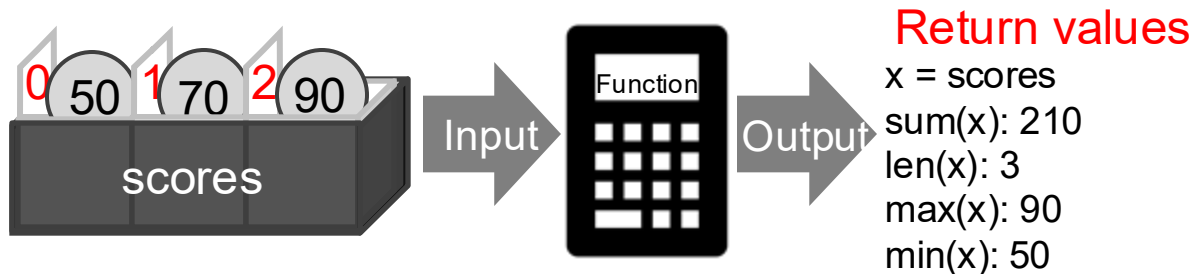
Functions for List Calculations



Built-in functions provided by Python related to list calculations.

Common Built-in Functions for Lists

Functions	Return Value
sum()	Total
len()	# of Element
max()	Maximum
min()	Minimum



Example Code

```
In [1]: scores = [50, 70, 90]
```

```
In [2]: num = len(scores)  
num
```

```
Out [2]: 3
```

Average =
Total / # of element

```
In [3]: sum(scores) / num
```

```
Out [3]: 70.0
```

```
In [4]: max(scores)
```

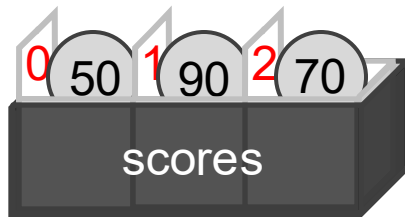
```
Out [4]: 90
```


A method is a function that belongs to an object.
You can call a method by appending a dot (.) to the variable name.

Function vs Method

Function : `min(scores)` → 50

Method : `scores.index(50)` → 0



*For the sort method, you must include parentheses () even if no arguments are provided. Note that it does not return a value; it modifies the list in place.

Example Code

```
In [1]: scores = [50, 90, 70]
        min(scores)
```

function

```
Out [1]: 50
```

method

```
In [2]: scores.index(50)
```

```
Out [2]: 0
```

method

```
In [3]: scores.sort()
        scores
```

```
Out [3]: [50, 70, 90]
```

Quiz - Check Your Understanding



Fill in the blanks.

Example Code (Lists)

```
In [1]: scores = [50, 70, 0]
        scores[1]
```

Out [1]:

```
In [2]: score_table = [['a', 'b', 'c'], scores]
        score_table[0]
```

Out [2]:

Example Code (Functions & Methods)

```
In [3]: sum(scores) / len(scores)
```

Out [3]:

```
In [4]: min(scores)
```

Out [4]:

```
In [5]: min_i = scores.index(min(scores))
        score_table[0][min_i]
```

Out [5]:

Quiz - Check Your Understanding



Answers

Example Code (Lists)

```
In [1]: scores = [50, 70, 0]
        scores[1]
```

```
Out [1]: 70
```

```
In [2]: score_table = [['a', 'b', 'c'], scores]
        score_table[0]
```

```
Out [2]: ['a', 'b', 'c']
```

	Col.[0]	Col.[1]	Col.[2]
Row[0]	['a',	'b',	'c',
Row[1]	[50,	70,	0]]

Example Code (Functions & Methods)

```
In [3]: sum(scores) / len(scores)
```

```
Out [3]: 40.0
```

Average =
Total / # of
Element

```
In [4]: min(scores)
```

```
Out [4]: 0
```

Minimum

Minimum
Values Index

```
In [5]: min_i = scores.index(min(scores))
        score_table[0][min_i]
```

```
Out [5]: 'c'
```

An object is a combination of values (**attributes**) and processes (**methods**). It is defined by a **class**.

Example Class and Objects

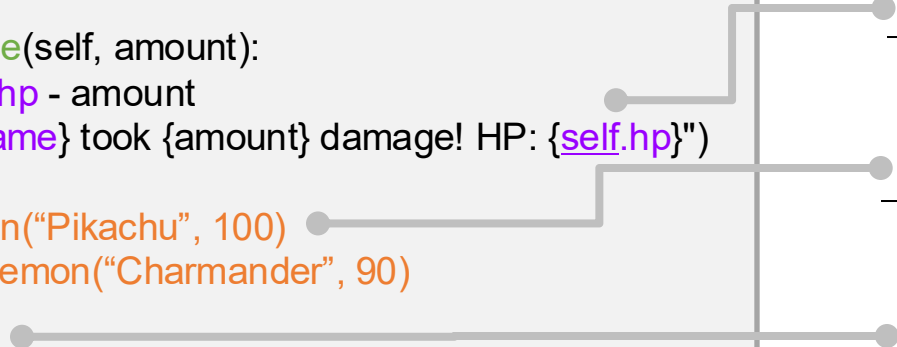
In [1]:

```
class Pokemon:
    def __init__(self, name, hp):
        self.name = name
        self.hp = hp

    def take_damage(self, amount):
        self.hp = self.hp - amount
        print(f"{self.name} took {amount} damage! HP: {self.hp}")

pikachu = Pokemon("Pikachu", 100)
charmander = Pokemon("Charmander", 90)

pikachu.take_damage(10)
```



Out [1]: Pikachu took 10 damage! HP: 90

The constructor method

`__init__` . Called when object is created

What is `self`?

The current object.

Access Attributes

`variable_name.attribute_name`

Create an Object (Instance)

`variable_name = ClassName()`

How to Use Methods

`variable_name.method_name()`

A **library** is a collection of Python program files provided by various organizations. To use the files in a library, you need to **import** them.

Libraries Used in Data Science



Library	Main Usage
NumPy	Fast Numerical Calculations
pandas	Handling Table Data
matplotlib	Data Visualization
scikit-learn	Building and Evaluating ML Models

Libraries help us conduct data analysis with just a few lines of codes!

Example Code

```
In [1]: import numpy as np
import pandas as pd

In [2]: np.pi

Out[2]: 3.141592653589793

In [3]: import matplotlib.pyplot as plt

In [4]: from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

Import

Accessing pi variable from NumPy with the .

Importing part of program from Scikit-learn

*There are many ways to do this, but no need to memorize them—copy and paste is totally fine!